Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ЛАБОРАТОРНАЯ 4

По дисциплине Контейнеризация и оркестрация приложений

Тема работы Docker Compose: оркестрация сервисов с Redis-кэшированием

Обучающийся Дощенников Никита Андреевич

Факультет Прикладной информатики

Группа К3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	_____	_____	Дощенников Н.А.
	(дата)	(подпись)	(Ф.И.О.)
Руководитель	_____	_____	Аминов Н.С.
	(дата)	(подпись)	(Ф.И.О.)

Санкт-Петербург
2026

Содержание

Персональные параметры 3
Этап 1. Подготовка проекта для Docker Compose 4
Задание 1.1 4
Задание 1.2 4
Задание 1.3 5
Этап 2. Добавление Redis для кеширования погоды 7
Задание 2.1 7
Задание 2.2 ??? 7
Задание 2.3 7
Этап 3. Запуск и проверка полного стека 8
Задание 3.1 8
Задание 3.2 8
Задание 3.3 9
Задание 3.4 10
Этап 4. Масштабирование сервиса 12
Задание 4.1 12
Задание 4.2 13
Задание 4.3 14
Задание 4.4 15
Этап 5. Финальный артефакт 17
Задание 5.1 17
Приложение 18

Персональные параметры

Параметр	Значение
Номер ИСУ	*465797*
Персональный порт (app)	*5527*
Персональный порт (Redis)	*6467*
Персональный префикс	*doschennikov*
Секретный ключ	*secret97doschennikov*
Имя базы данных	*doschennikov97*

Таблица 1. Персональные параметры

Этап 1. Подготовка проекта для Docker Compose

Задание 1.1

Я создал файл `.env` с персональными переменными:

```
APP_OWNER=doschennikov
APP_SECRET=secret97doschennikov
APP_PORT=5527
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_USER=user_465797
POSTGRES_PASSWORD=pgpass_465797
POSTGRES_DB=assistant_465797
PG_PORT=5527
REDIS_PORT=6467
CACHE_TTL=120
```

Задание 1.2

Я создал `docker-compose.yml` с сервисами app и db:

```yaml
# version: '3.8'
services:
  db:
    container_name: doschennikov-postgres
    image: postgres:15
    networks:
      - doschennikov-backend
    env_file:
      - .env
  app:
    build: .
    container_name: doschennikov-assistant
    networks:
      - doschennikov-backend
    depends_on:
      db:
        condition: service_healthy
    env_file:
      - .env
    ports:
      - "5527:5527"

volumes:
  doschennikov-pgdata:

networks: doschennikov-backend:
```

```
    driver: bridge
    name: doschennikov-backend
```

Задание 1.3

Затем проверил базовый запуск:

```
docker-compose up -d --build
```

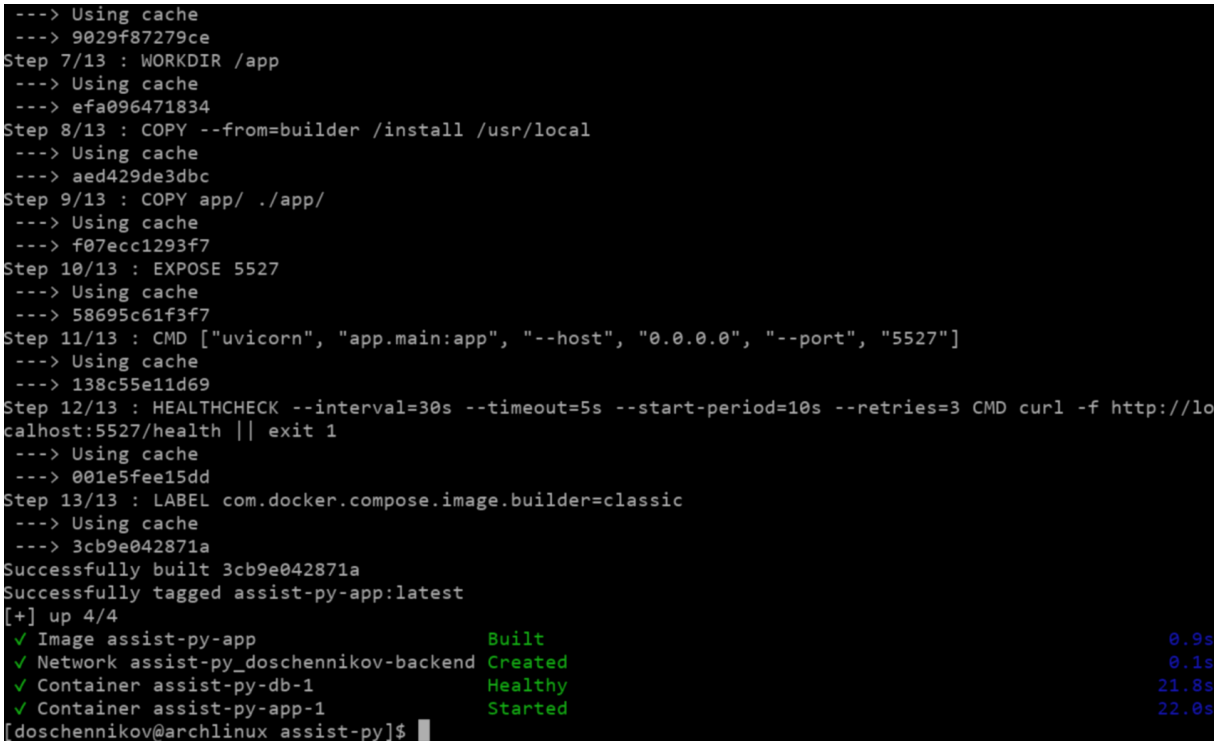


Рисунок 1. Проверка базового запуска: сборка

```
docker-compose ps
```

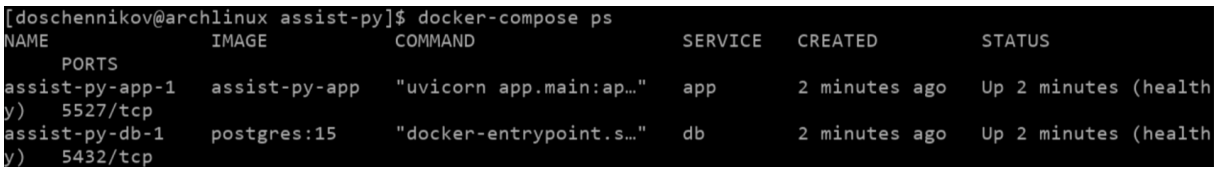


Рисунок 2. Проверка базового запуска: контейнеры

```
docker-compose logs app
```

```
[doschennikov@archlinux assist-py]$ docker-compose logs app
app-1  | INFO:     Started server process [1]
app-1  | INFO:     Waiting for application startup.
app-1  | WARNING:  Redis unavailable (Error 111 connecting to localhost:6379. Connection refused.) — cach
ing disabled
app-1  | INFO:     Application startup complete.
app-1  | INFO:     Uvicorn running on http://0.0.0.0:5527 (Press CTRL+C to quit)
app-1  | INFO:     127.0.0.1:40896 - "GET /health HTTP/1.1" 200 OK
app-1  | INFO:     127.0.0.1:44658 - "GET /health HTTP/1.1" 200 OK
app-1  | INFO:     127.0.0.1:60750 - "GET /health HTTP/1.1" 200 OK
app-1  | INFO:     127.0.0.1:51048 - "GET /health HTTP/1.1" 200 OK
app-1  | INFO:     127.0.0.1:33610 - "GET /health HTTP/1.1" 200 OK
app-1  | INFO:     127.0.0.1:57986 - "GET /health HTTP/1.1" 200 OK
[doschennikov@archlinux assist-py]$
```

Рисунок 3. Проверка базового запуска: логи `app`

```
docker-compose logs db
```

```
db-1   | server started
db-1   | CREATE DATABASE
db-1   |
db-1   |
db-1   | /usr/local/bin/docker-entrypoint.sh: ignoring /docker-entrypoint-initdb.d/*
db-1   |
db-1   | waiting for server to shut down...2026-10-08 08:09:39.279 UTC [61] LOG:  received fast shutdown r
equest
db-1   | .2026-10-08 08:09:39.291 UTC [61] LOG:  aborting any active transactions
db-1   | 2026-10-08 08:09:39.295 UTC [61] LOG:  background worker "logical replication launcher" (PID 67)
exited with exit code 1
db-1   | 2026-10-08 08:09:39.295 UTC [62] LOG:  shutting down
db-1   | 2026-10-08 08:09:39.307 UTC [62] LOG:  checkpoint starting: shutdown immediate
db-1   | ..2026-10-08 08:09:41.560 UTC [81] FATAL:  the database system is shutting down
db-1   | .2026-10-08 08:09:42.957 UTC [62] LOG:  checkpoint complete: wrote 922 buffers (5.6%); 0 WAL file
(s) added, 0 removed, 0 recycled; write=0.113 s, sync=3.457 s, total=3.663 s; sync files=301, longest=0.0
19 s, average=0.012 s; distance=4239 kB, estimate=4239 kB
db-1   | 2026-10-08 08:09:42.979 UTC [61] LOG:  database system is shut down
db-1   |  done
db-1   | server stopped
db-1   |
db-1   | PostgreSQL init process complete; ready for start up.
db-1   |
db-1   | 2026-10-08 08:09:43.059 UTC [1] LOG:  starting PostgreSQL 15.19 (Debian 15.19-1.pgdg13+2) on x86_
64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
db-1   | 2026-10-08 08:09:43.060 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
db-1   | 2026-10-08 08:09:43.060 UTC [1] LOG:  listening on IPv6 address "::", port 5432
db-1   | 2026-10-08 08:09:43.085 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432
"
db-1   | 2026-10-08 08:09:43.112 UTC [85] LOG:  database system was shut down at 2026-10-08 08:09:42 UTC
db-1   | 2026-10-08 08:09:43.134 UTC [1] LOG:  database system is ready to accept connections
[doschennikov@archlinux assist-py]$
```

Рисунок 4. Проверка базового запуска: логи `db`

Затем я убедился, что оба контейнера запущены и приложение отвечает:

```
curl http://localhost:5527/
curl http://localhost:5527/health
```

```
[doschennikov@archlinux assist-py]$ curl http://localhost:5527/
{"message":"👋 Привет, doschennikov!","service":"personal-assistant","version":"3.0","endpoints":["/me","
/notes","/weather/{city}","/secret/{key}","/cache/stats"],"hostname":"d568de0f017b","server_time":"2026-1
0-08T08:22:45.887999+00:00"}[doschennikov@archlinux assist-py]$ curl http://localhost:5527/health
{"status":"healthy","owner":"doschennikov","redis":false,"hostname":"d568de0f017b","server_time":"2026-10
-08T08:22:47.576269+00:00"}[doschennikov@archlinux assist-py]$
```

Рисунок 5. Проверка работоспособности контейнеров

Этап 2. Добавление Redis для кеширования погоды

Задание 2.1

Я добавил новый сервис `redis` в `docker-compose.yml`, при этом приложение (`app`) зависит от Redis.

```
redis:
  image: redis:7-alpine
  container_name: doschennikov-redis
  ports:
    - "${REDIS_PORT}:6379"
  networks:
    - doschennikov-backend
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5
```

Также я обновил секцию `depends_on` у сервиса `app`, добавив зависимость от `redis`:

```
services:
  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
```

Задание 2.2 ???

Задание 2.3

Я убедился, что в `main.py` интегрирован Redis.

Этап 3. Запуск и проверка полного стека

Задание 3.1

Я пересобрал и запустил все сервисы:

```
docker-compose down -v
docker-compose up -d --build
```

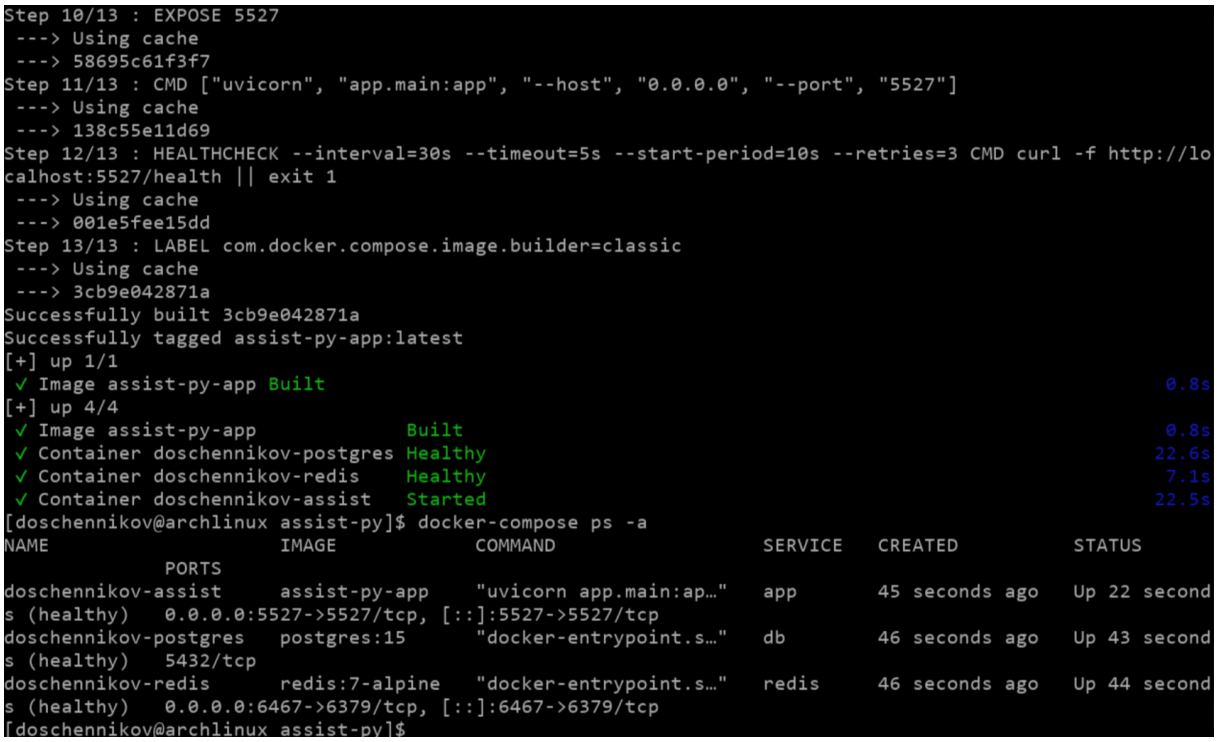


Рисунок 6. Сборка сервисов

Задание 3.2

Я проверил порядок запуска:

```
docker-compose logs --tail=30
docker-compose logs app | head -5
```

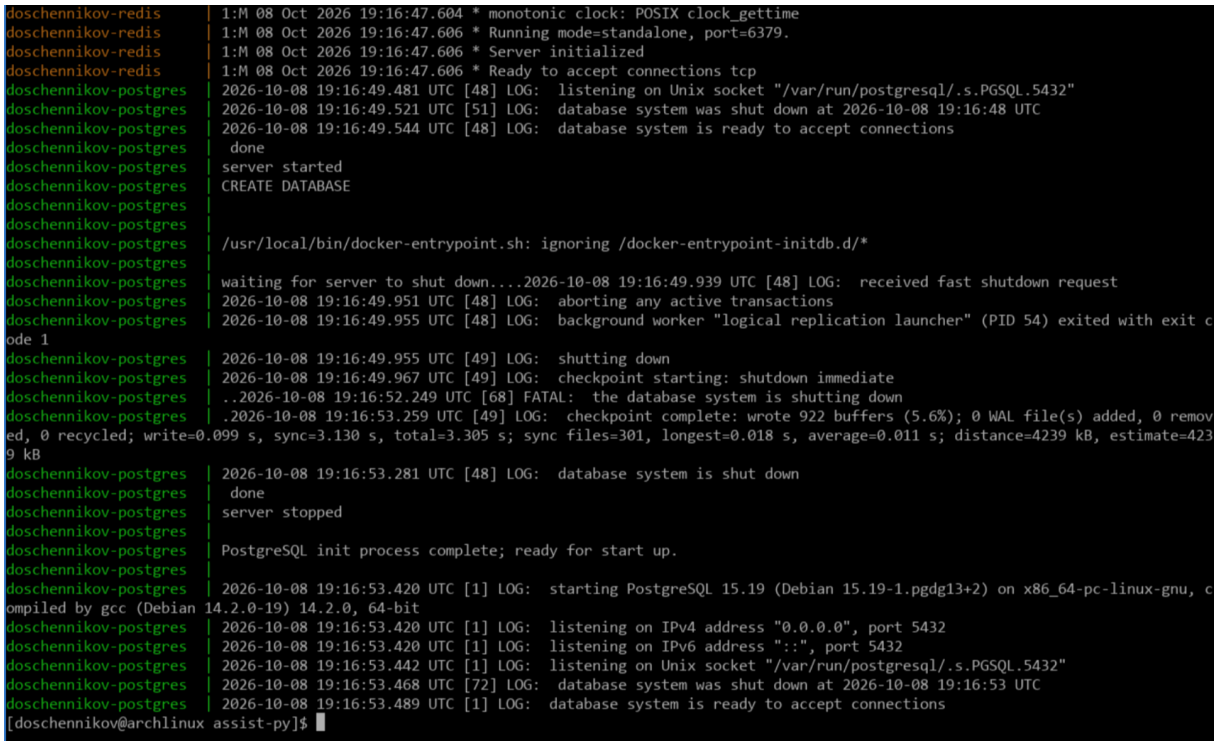


Рисунок 7. Порядок запуска (1)

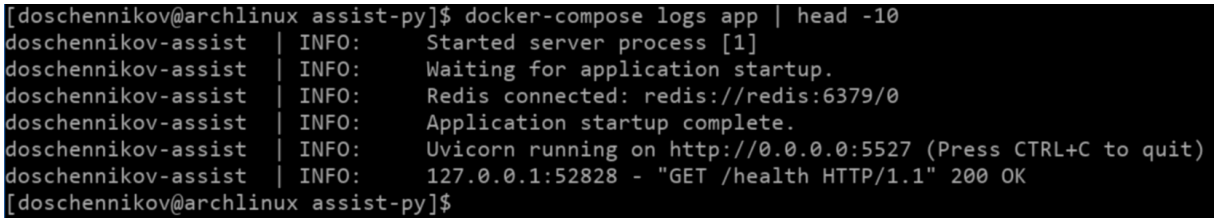


Рисунок 8. Порядок запуска (2)

Задание 3.3

Затем я проверил работу кеширования:

```
curl http://localhost:5527/weather/Moscow
curl http://localhost:5527/weather/Moscow
```



Рисунок 9. Данные из API и кэша

Затем я проверил статистику кэша:

```
curl http://localhost:5527/cache/stats
```

```
180842+00:00"}[doschennikov@vm assist-py]$ curl http://localhost:5527/cache/stats
{"owner":"doschennikov","cache_ttl_setting":120,"cached_cities":1,"keys":[{"key":"weather:
doschennikov:moscow","ttl_seconds":117}],"hostname":"292598e442e8","server_time":"2026-10-
09T17:16:15.801000+00:00"}[doschennikov@vm assist-py]$
```

Рисунок 10. Статистика кэша

Я проверил данные напрямую в Redis:

```
docker exec doschennikov-redis redis-cli KEYS "weather:*"
docker exec doschennikov-redis redis-cli TTL
weather:doschennikov:moscow
```

```
895712+00:00"}[doschennikov@vdocker exec doschennikov-redis redis-cli KEYS "weather:*"
weather:doschennikov:moscow
[doschennikov@vm assist-py]$ docker exec doschennikov-redis redis-cli TTL weather:doschenn
ikov:moscow
115
[doschennikov@vm assist-py]$
```

Рисунок 11. Данные в Redis

Задание 3.4

Я посмотрел список сетей:

```
docker network ls --filter "name=doschennikov"
```

```
[doschennikov@vm assist-py]$ docker network ls --filter "name=doschennikov"
NETWORK ID     NAME                   DRIVER    SCOPE
64f49c851231   doschennikov-backend   bridge    local
```

Рисунок 12. Список сетей

```
docker network inspect doschennikov-backend
```



Рисунок 13. Детали сети

Я посмотрел список томов:

```
docker volume ls --filter "name=doschennikov"
```

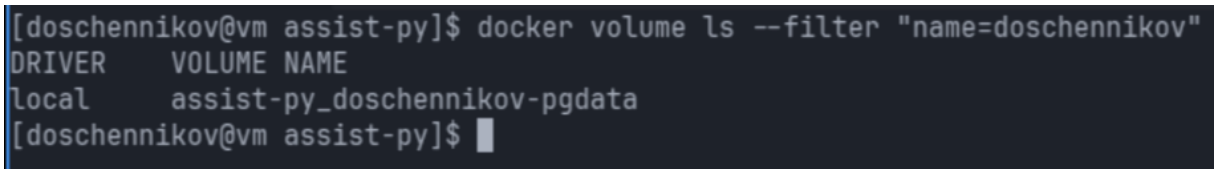


Рисунок 14. Список томов

Этап 4. Масштабирование сервиса

Задание 4.1

Я создал отдельный `docker-compose.scale.yml`. Для масштабирования я убрал `container_name` у `app` и изменил порт на диапазон:

```yaml
# docker-compose.scale.yml
# version: '3.8'

services:
  db:
    image: postgres:15
    container_name: doschennikov-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - doschennikov-pgdata:/var/lib/postgresql/data
    networks:
      - doschennikov-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: doschennikov-redis
    networks:
      - doschennikov-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "5527"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL:
        postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/
```

```
${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - doschennikov-backend

volumes:
  doschennikov-pgdata:

networks:
  doschennikov-backend:
  driver: bridge
```

Задание 4.2

Я запустил масштабирование. Для этого я остановил текущие контейнеры:

```
docker-compose down
```

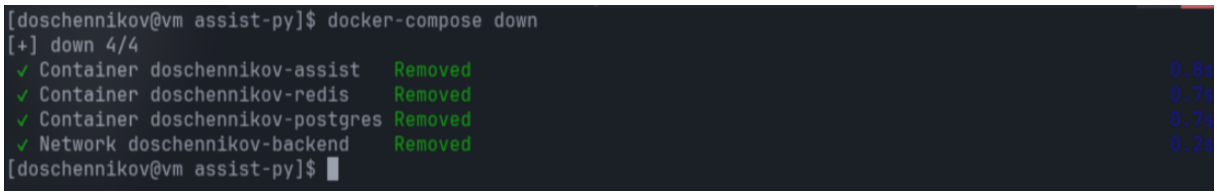


Рисунок 15. Остановка контейнеров

Затем запустил с 3 экземпляра `app`:

```
docker-compose -f docker-compose.scale.yml up -d --build --scale app=3
```

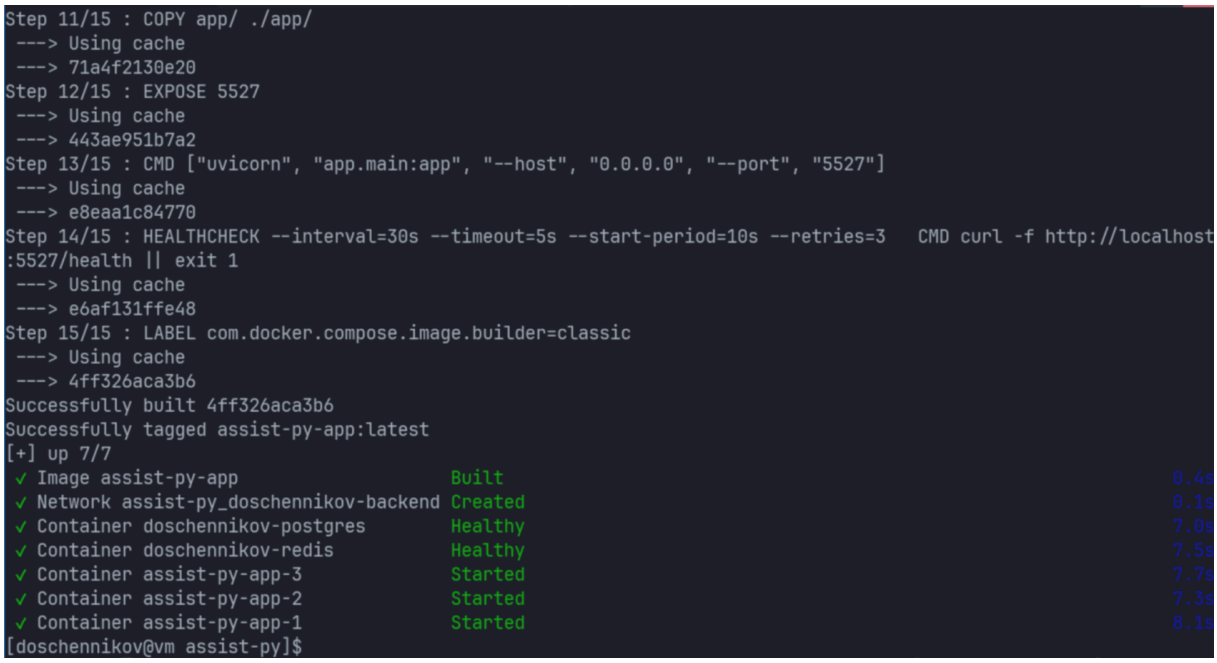


Рисунок 16. Запуск с масштабированием

Я также проверил, что запустилось 3 экземпляра:

```
docker-compose -f docker-compose.scale.yml ps
```

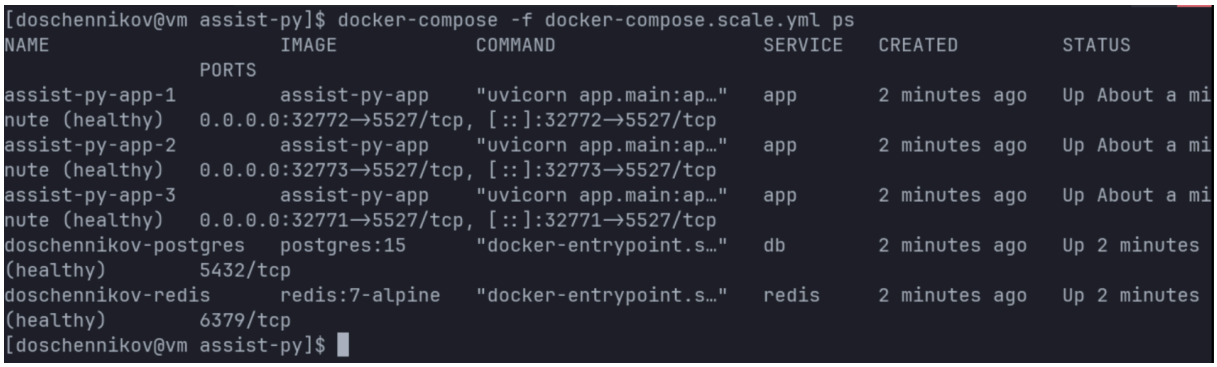


Рисунок 17. Проверка запуска

Задание 4.3

Я проверил работу масштабированных экземпляров. Для этого я узнал назначенные порты:

```
docker-compose -f docker-compose.scale.yml ps app
```

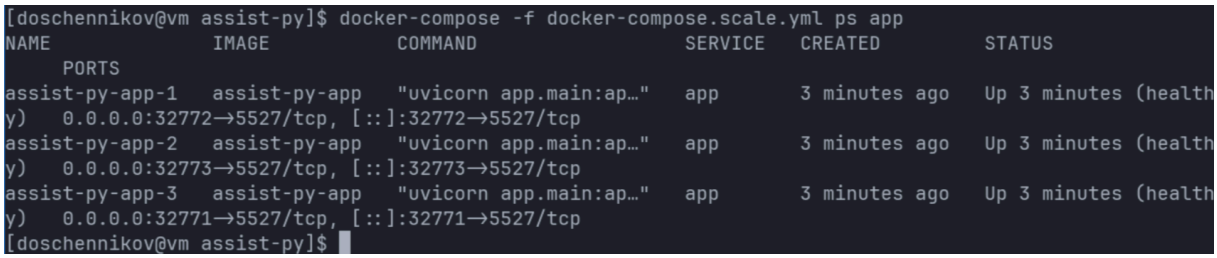


Рисунок 18. Назначенные порты

Затем я обратился к каждому экземпляру:

```
curl http://localhost:32771/health
curl http://localhost:32772/health
curl http://localhost:32773/health
```

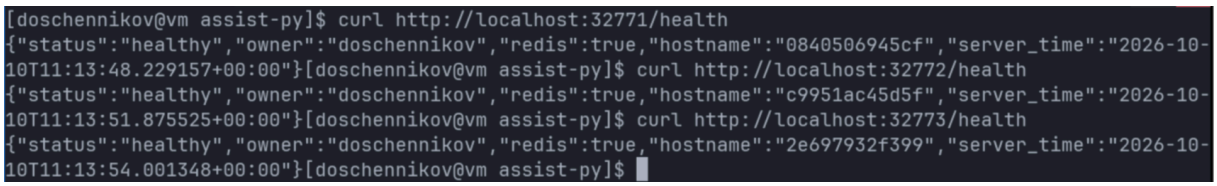


Рисунок 19. Healthcheck

Затем я проверил, что кэш Redis общий для всех экземпляров:

```
curl http://localhost:32771/weather/London
curl http://localhost:32772/weather/London
curl http://localhost:32773/cache/stats
```



Рисунок 20. Общий кэш для нескольких экземпляров

Задание 4.4

Я вернулся к основной конфигурации. Для этого я остановил масштабированные контейнеры:

```
docker-compose -f docker-compose.scale.yml down
```

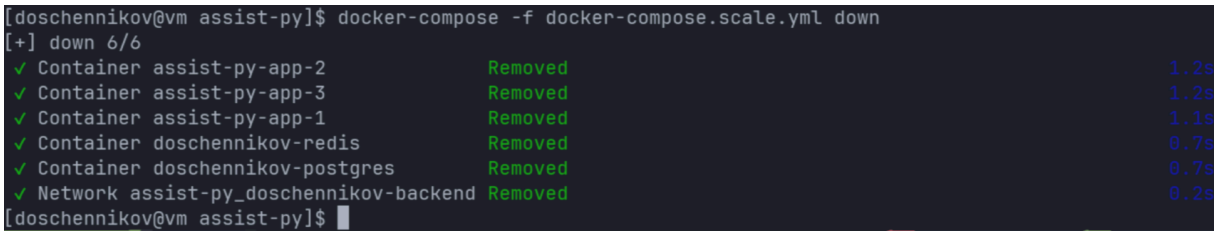


Рисунок 21. Остановка контейнеров

А затем запустил основную конфигурацию:

```
docker-compose up -d
```

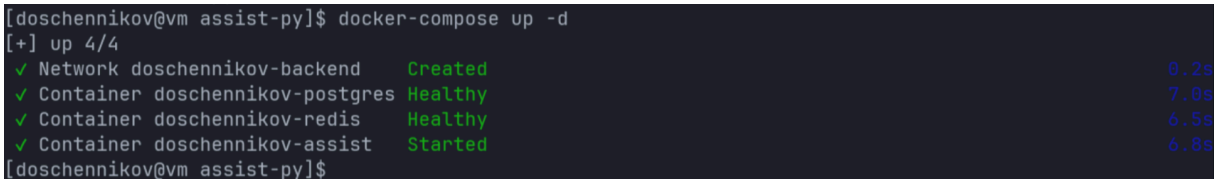


Рисунок 22. Запуск основной конфигурации

Этап 5. Финальный артефакт

Задание 5.1

Я сформировал финальный артефакт:

```
echo "========================================" && \
echo "ЛР4 завершена: $(date '+%d.%m.%Y %H:%M:%S')" && \
echo "Студент: $(whoami)@$(hostname)" && \
echo "========================================" && \
echo "" && \
echo "Docker Compose сервисы:" && \
docker-compose ps && \
echo "" && \
echo "Сети:" && \
docker network ls --filter "name=doschennikov" && \
echo "" && \
echo "Volumes:" && \
docker volume ls --filter "name=doschennikov" && \
echo "" && \
echo "Кэш Redis:" && \
docker exec doschennikov-redis redis-cli KEYS "weather:*" && \
echo "" && \
echo "Погода (из кэша):" && \
curl -s http://localhost:5527/cache/stats | python3 -m json.tool
```

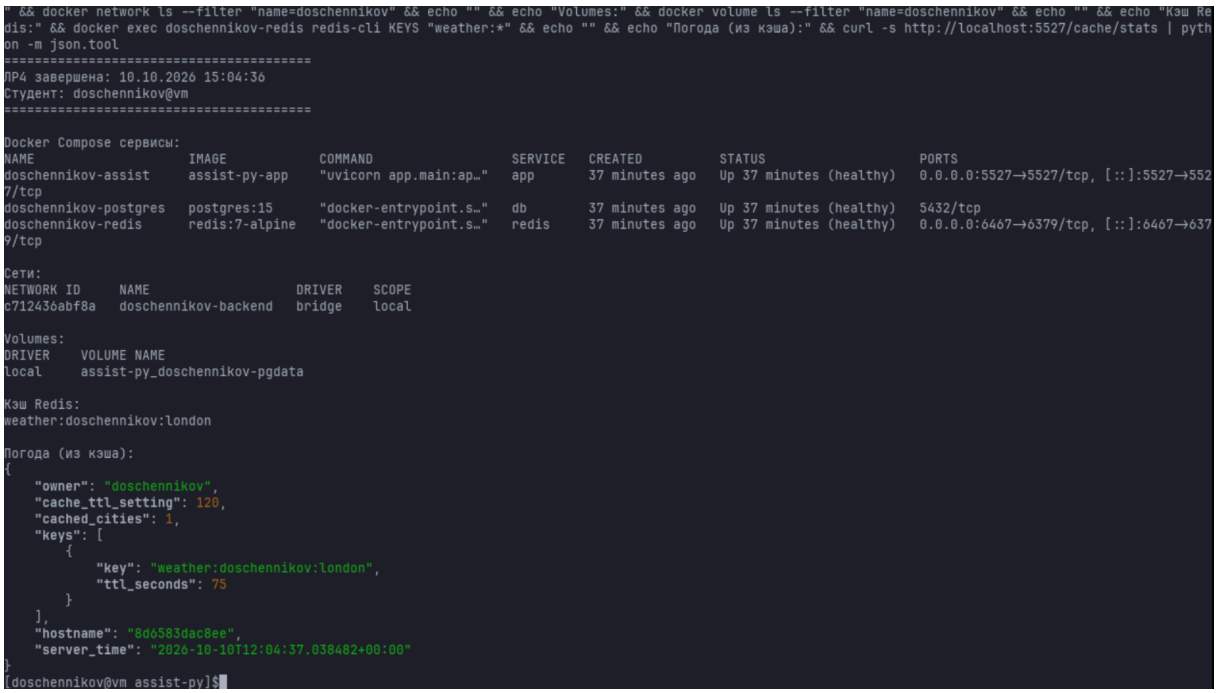


Рисунок 23. Финальный артефакт

Приложение

`.dockerignore`:

```
__pycache__/
*.pyc
*.pyo
.git
.gitignore
.venv/
venv/
.env
*.log
Dockerfile*
README.md
```

`.env`:

```
APP_OWNER=doschennikov
APP_SECRET=secret97doschennikov
APP_PORT=5527
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_USER=user_465797
POSTGRES_PASSWORD=pgpass_465797
POSTGRES_DB=assistant_465797
PG_PORT=5527
REDIS_PORT=6467
CACHE_TTL=120
```

`docker-compose.scale.yml`:

```
services:
  db:
    image: postgres:15
    container_name: doschennikov-postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - doschennikov-pgdata:/var/lib/postgresql/data
    networks:
        - doschennikov-backend
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    container_name: doschennikov-redis
    networks:
        - doschennikov-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "5527"
    environment:
      APP_OWNER: ${APP_OWNER}
      APP_SECRET: ${APP_SECRET}
      DATABASE_URL:
        postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/
${POSTGRES_DB}
      REDIS_URL: redis://redis:6379/0
      CACHE_TTL: ${CACHE_TTL}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
```

```
    networks:
      - doschennikov-backend

volumes:
  doschennikov-pgdata:

networks:
  doschennikov-backend:
    driver: bridge
```

`docker-compose.yml`:

```
services:
  db:
    container_name: doschennikov-postgres
    image: postgres:15
    networks:
      - doschennikov-backend
    env_file:
      - .env
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d $
{POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
    volumes:
      - doschennikov-pgdata:/var/lib/postgresql/data

  app:
    build: .
    container_name: doschennikov-assist
    networks:
      - doschennikov-backend
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    env_file:
      - .env
    environment:
      REDIS_URL: redis://redis:6379/0
    ports:
      - "5527:5527"

  redis:
    image: redis:7-alpine
    container_name: doschennikov-redis
    ports:
      - "${REDIS_PORT}:6379"
    networks:
      - doschennikov-backend
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
```

```
      retries: 5

volumes:
  doschennikov-pgdata:

networks:
  doschennikov-backend:
    driver: bridge
    name: doschennikov-backend
```

`Dockerfile`:

```
FROM python:3.11 AS builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim

LABEL maintainer="doschennikov@lab.local"
LABEL version="1.0"

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends curl
\
  && rm -rf /var/lib/apt/lists/*

COPY --from=builder /install /usr/local

COPY app/ ./app/

EXPOSE 5527

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "5527"]

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3
\
  CMD curl -f http://localhost:5527/health || exit 1
```